

Welcome!

Time	Mon	Tue	Wed	Thu	Fri
10:00–12:00	Lecture: Intro	Lecture: Waveshaping	Lecture: Delay/Modulation	Special Topics	Special Topics
12:00–13:00	Exercise 01	Exercise 03	Exercise 05	Project Work	Project Work
13:00–14:00	Lunch	Lunch	Lunch	Lunch	Lunch
14:00–16:00	Lecture: Filters	Lecture: Dynamics	Individual Projects	Project Work	Project Work
16:00–17:00	Exercise 02	Exercise 04	Project Work	Project Work	Project Presentations



|||≡ Ableton

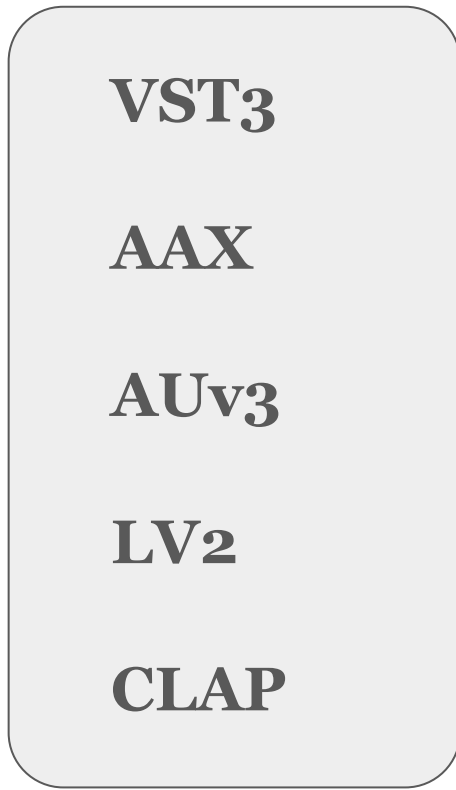
???



FL STUDIO



Logic Pro X



Plugin Standards

- Plugin standards are **protocols** for how a **plugin** and a **host** should “talk” to each other. Plugins are dynamic libraries; the host is usually a DAW.
- These standards are language-agnostic.
 - (yes, they’re usually defined in C, but once everything is compiled to bytecode, the computer doesn’t care what language you use as long as you follow the “contract”)
- The legacy formats are old, crusty, and painful to work with.

“Oh and by the way, VST support is incredibly time-consuming. The technology is basically a lawless territory for cowboys to go wild. You can’t just provide support for the standard, and expect everything to work. What ends up happening is that you discover hundreds of popular VSTs which fail for weird unique reasons ... it was like a PTSD flashback”

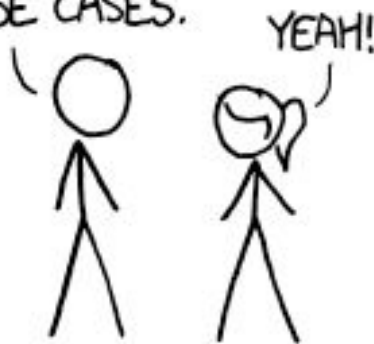
- Martin Keary (tantacrul)

HOW STANDARDS PROLIFERATE:

(SEE: A/C CHARGERS, CHARACTER ENCODINGS, INSTANT MESSAGING, ETC.)

SITUATION:
THERE ARE
14 COMPETING
STANDARDS.

14?! RIDICULOUS!
WE NEED TO DEVELOP
ONE UNIVERSAL STANDARD
THAT COVERS EVERYONE'S
USE CASES.



SOON:

SITUATION:
THERE ARE
15 COMPETING
STANDARDS.

CLAP: A modern plugin standard

- CLAP is a modern, open-source plugin standard.
- It's new, but seeing rapid adoption.
- It's much easier to write both plugins *and* hosts (DAWs) with CLAP.
- CLAP also has lots of cool features for both plugins and hosts:
 - Polyphonic parameter automation
 - Thread pools
 - Preset management & navigation
 - And more!
- CLAP plugins can target VST3 and AU too!

What about JUCE?

- Industry standard (ish)... but most big players don't use it.
- JUCE started in 2006. There are a LOT of things in JUCE that are redundant in 2026.
- CLAP has a lot of forward thinking stuff: polyphonic modulation, thread pools, etc etc.
- Going “one level lower” is incredibly instructive. You learn a lot about how audio is processed on computers... like a “real engineer”
- At a certain point, the JUCE isn't worth the squeeze:
 - Most “serious plugins” don't use the DSP in JUCE
 - The UI system is 20+ years old (no GPU support) lots of weird quirks (it re-implements half the STL), tons legacy crust, forces weird JUCE-isms upon the developer
 - You're really just paying (JUCE isn't free) for the wrapper that turns your plugin into a VST3/AU/AAX
 - This is particularly relevant in the age of cheap code from LLMs (more on that later)



It's all just float buffers...

- CLAP? VST3? JUCE? AAX? This all might sound overwhelming. **It isn't!**
- All plugins work in the same way, more or less.
- Once we understand the **principles** of:
 - Processing blocks of audio samples
 - Managing the lifecycle of our plugin
 - Basic digital signal processing (DSP)
 - Best practices for writing audio code
- ..then the choice of language/platform/whatever is decoration.
- Audio code is literally just **floating-point arithmetic**. That's it.

The LLM-ephant in the room

“Claude, build Serum 3; make no mistakes.”

This “platform-agnostic” approach is easier than ever in the age of LLMs.

Some have argued that software engineering is dead.

Why study audio plugin engineering when a LLM agent can make plugins for us?

A Nightmare on LLM Street

- LLM agents are great for small prototypes. They struggle to write human-comprehensible, extensible codebases for large, complex ideas.
 - Eventually, you'll be limited not by your dreams or talent but *by the quality of your LLM*.
- DSP code is tricky (as we'll discuss). Lots of ways to shoot yourself in the foot.
- Learning cool stuff is fun!
- If we outsource our complex thoughts to LLMs, **we lose the ability to think about complex things.**

Using LLMs the right way

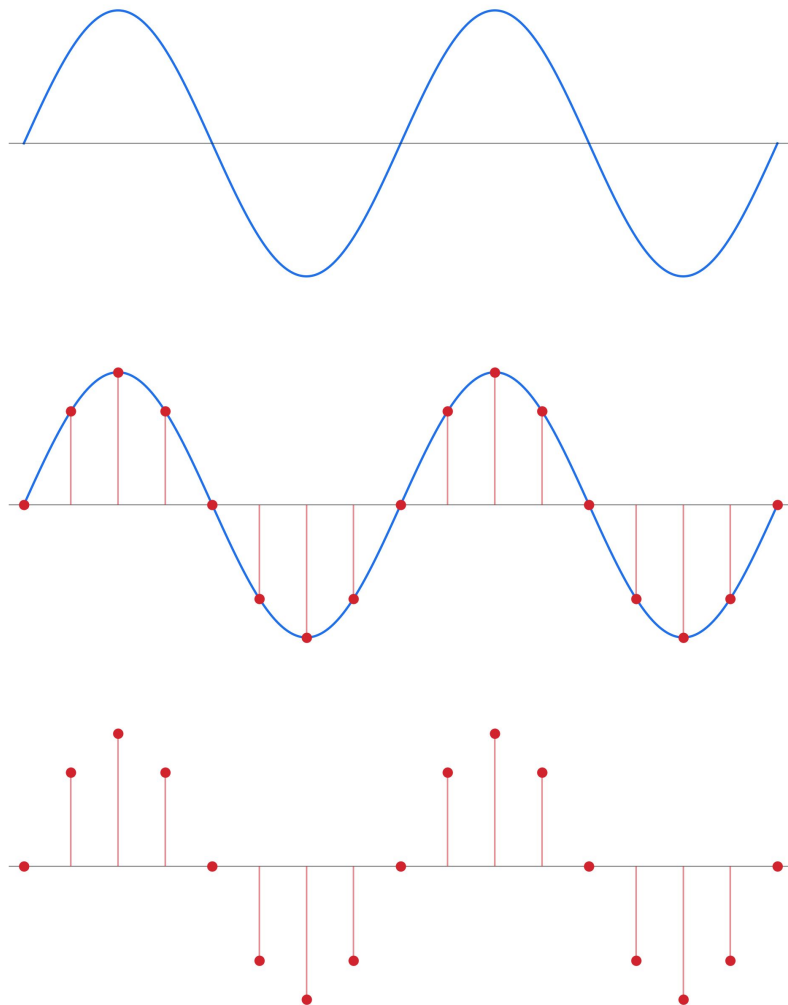
We encourage LLMs and agents to the extent that they aid, not supplant, the learning process.

- Use LLMs to learn about codebases, understand syntax, and get un-stuck.
- Use LLMs to learn about DSP techniques!
- Use LLMs to help with annoying “setup” problems
- **We will not be able to cover very much DSP in one week. LLMs will help you self-study.**
- Don't use LLMs to write all your code. Try writing your code by hand first.

“I hear, and I forget. I see, and I remember. I do, and I understand.”

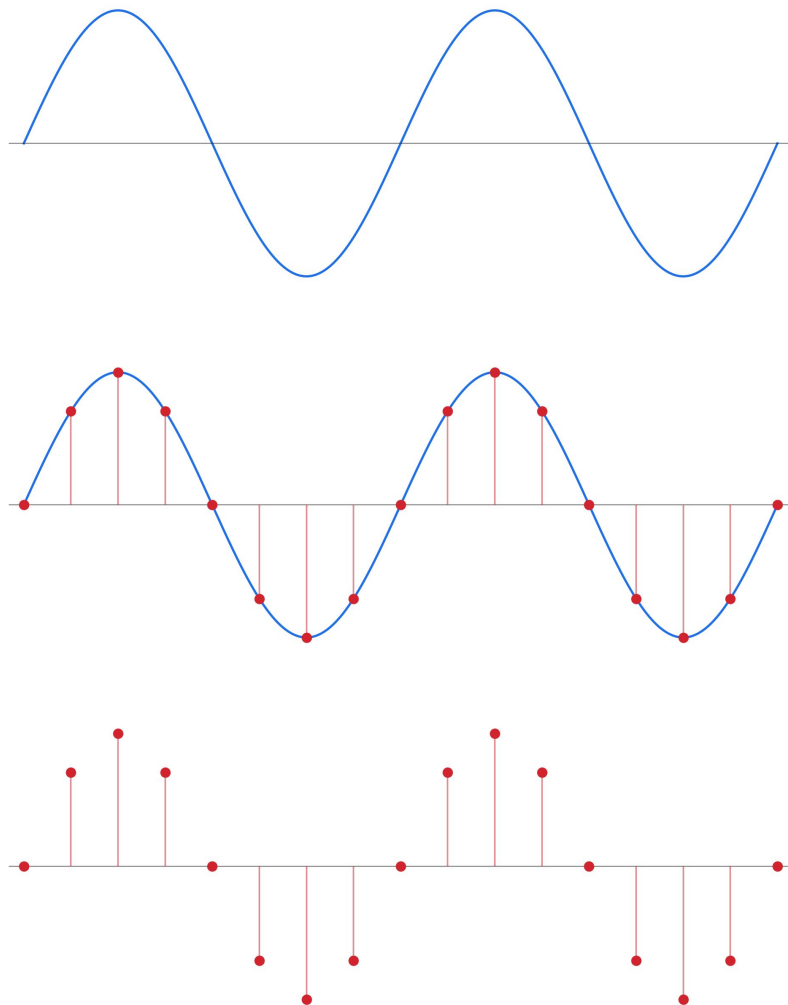
Digital Audio Basics

- Real world samples are continuous.
- Computers *sample* real world signals.
- A signal is represented by a buffer of floating point values.
- By convention, samples are in $[-1,1]$
- The **sampling rate** is given in Hertz, which is the number of samples per second.
 - 44,100 Hz (44.1 KHz) is very common



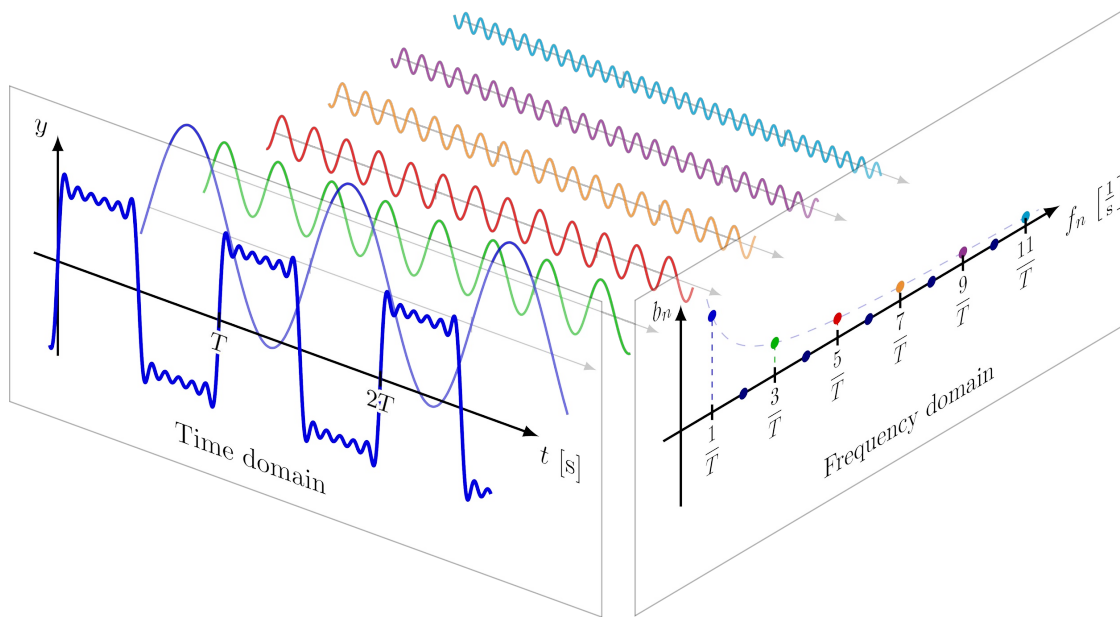
Digital Audio Basics

- Real world samples are continuous.
- Computers *sample* real world signals.
- A signal is represented by a buffer of floating point values.
- By convention, samples are in $[-1,1]$
- The **sampling rate** is given in Hertz, which is the number of samples per second.
 - 44,100 Hz (44.1 KHz) is very common



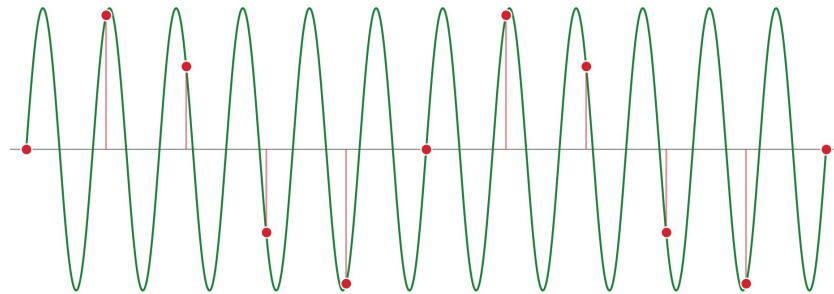
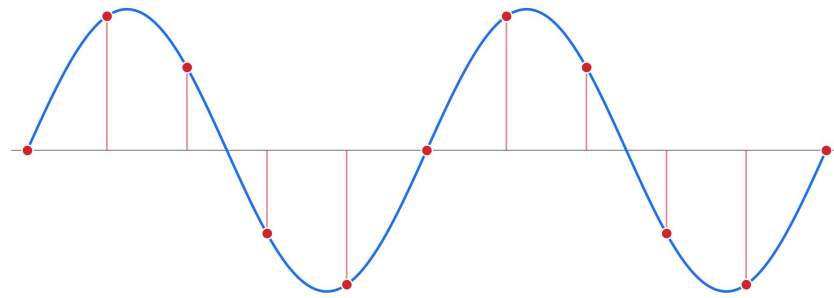
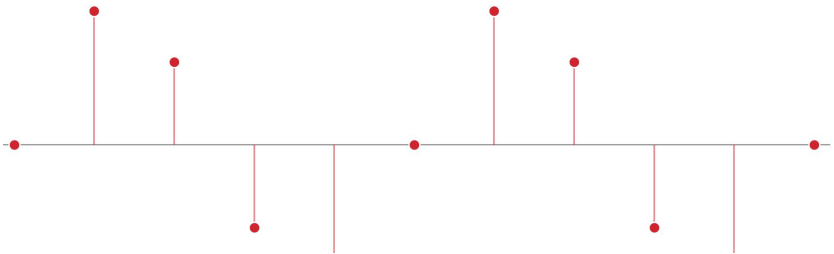
Digital Audio Basics

- Sine waves, or *sinusoids*, are the building blocks of signals
- Any periodic signal can be written as a sum of sinusoids.



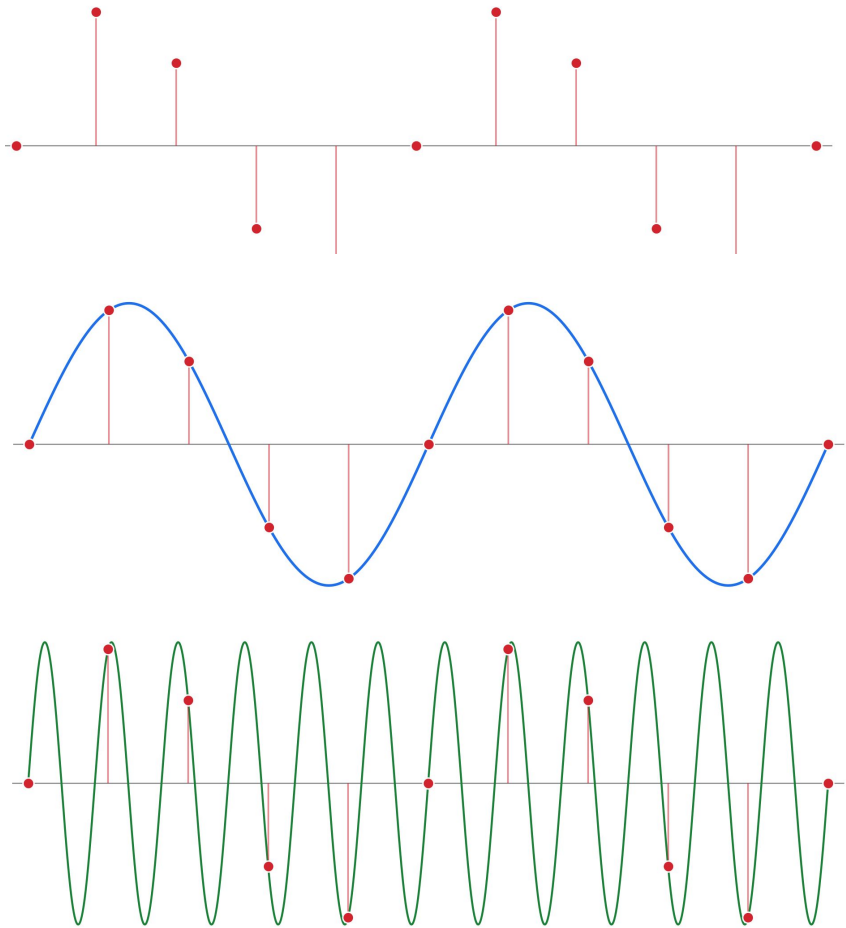
Aliasing

- Our choice of sampling rate is important.
- A sampled signal is **ambiguous**.
- We can fit sine waves of different frequencies onto the same sampled signal.
- We call this **aliasing**.
- When does aliasing occur?



Aliasing

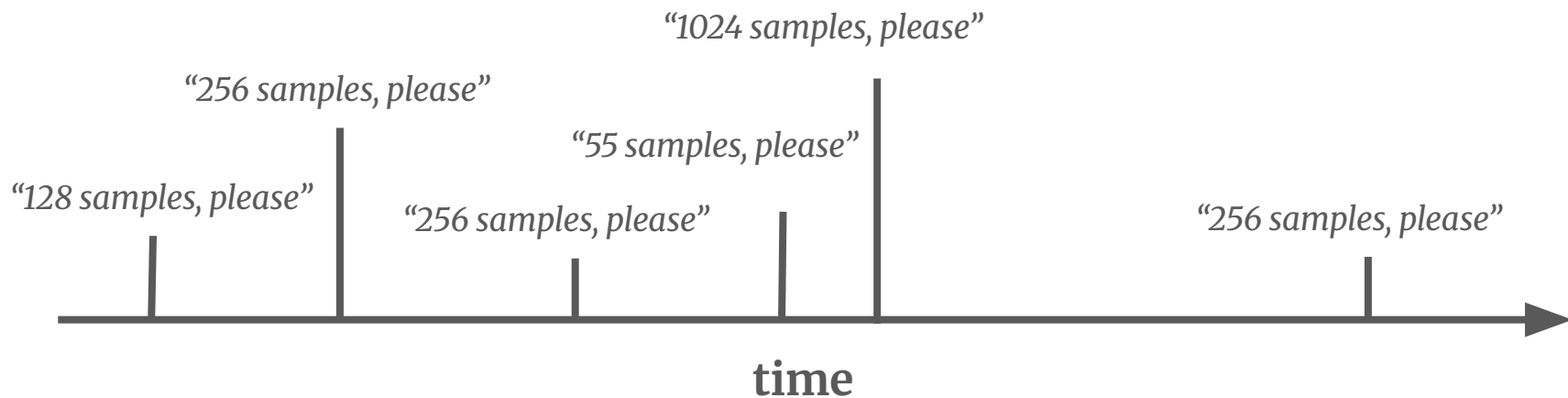
- If we choose sampling rate F , we can only sample frequencies up to $F/2$.
- Frequencies below $F/2$ are **losslessly captured**.
- This is the **Nyquist-Shannon sampling theorem**.
- 44.1 kHz audio contains frequencies up to ~ 22 kHz – the limit of human hearing.



How do plugins process audio?

- Plugin audio processing is **real time**. Your plugin **must** process a certain number of samples per unit time.
- Example: stereo effect at 44.1 kHz -> 88,200 samples/second!
- If you take too long to process samples, the universe explodes.
- You're sharing the audio context with many other plugins, so you should process samples with **plenty** of time to spare.

Writing (good) audio code is like writing code for a Mars lander. Every byte counts.



The host (DAW) will request a **block of samples** by calling a **process** function in your plugin.

It may request blocks of various sizes, but the total number of samples/sec always corresponds to the sampling rate.

The process Function

```
function process(ProcessInfo info):  
    events = info.events  
    for each event in events:  
        if event is a parameter_update:  
            updateParameter(event)  
        if event is MIDI:  
            processMIDI(event)  
        if event is ... :  
            processSomethingElse(event)  
  
    input_samples = info.in  
    processed_samples = doDspStuff(input_samples)  
    input.out = processed_samples
```

The **process** function is our “main function”.

Each time it is called we must process a **block of audio samples**.

We also must process **events**. Each event is **timestamped** with the sample when the event occurred.

No matter which language/standard/framework you use, your process block will look something like this!

The init Function

```
function init():  
    loadPresets()  
    displayWelcomeMessage()  
    somethingElse()  
    . . .
```

The **init** function is called **once** when plugin is instantiated.

This is basically your plugin's constructor. Do whatever setup you need to do here.

However, we **don't know the block size/sample rate/channel configuration** yet.

The activate Function

```
function activate(sampleRate,  
                 minFrameSize,  
                 maxFrameSize):  
    resetEverything()  
    allocateBuffers(maxFrameSize)  
    setupDSP(sampleRate)
```

The **activate** function is called by the host when the **audio system is configured**.

The host gives us a sample rate, min/max block size, and any other relevant information for audio processing.

Here, we might:

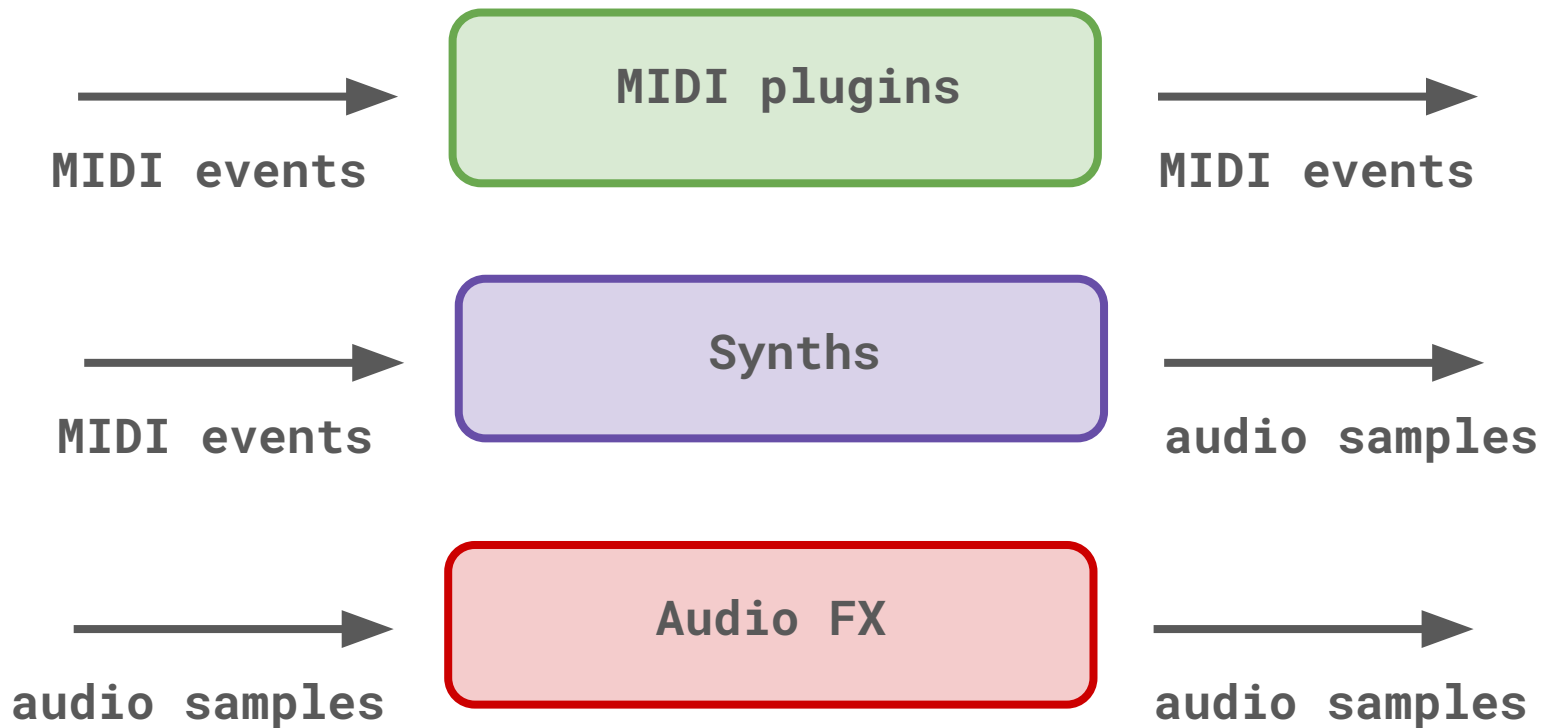
- Allocate memory based on the block size
- Prepare our DSP pipeline

This function can be called multiple times during a plugin's life-cycle!

Putting the “fun” in function

- There are lots of other functions you might want to implement, such as:
 - MIDI/Audio port configuration
 - Parameter registration
 - Saving/loading plugin state
 - And many more!
- We can talk about these as we go. Try to find some functions like these in the example code and see if you can infer how they work.

We classify plugin types by what sorts of musical data they consume and produce.



Plugin Standards are Language Agnostic

Our example code is given in four different languages:

- C
- C++ (*most popular*)
- Rust
- Jai

You can use **any programming language you want** in this workshop*

Plugin Standards are Language Agnostic

Our example code is given in four different languages:

- C
- C++ (*most popular*)
- Rust
- Jai

You can use **any programming language you want** in this workshop*

**as long as it can produce a native shared library that can interface with C-style function pointers... Zig, D, Swift, Ada...*

...even Python (please don't, though?)

Exercise 1: A Gain plugin

- Clone the workshop GitHub repo
- See if you can get the example plugin to build.
- Study the example code. Try to identify the **init** function, **activate** function, and **process** function.
 - The code might look more verbose than our pseudocode in these slides, since it's real code. Nothing to be afraid of!
- Get comfortable in your development environment. Make sure you can easily compile your plugin, get it running in your plugins folder, and load it into your DAW.
 - The **clap-host** is a nice lightweight host you can use for debugging, but a production DAW works too.
 - For DAWs, use **Reaper** or **Bitwig** (free version)
- Using the **01_generator** template, write an audio effect plugin that **changes the gain** of the input audio. To do this, you'll need to multiply all the incoming samples by some incoming scalar, and then write them to the output buffer.

Have fun!